

Test plan: how the captured logs were taken

Runs A to H: stock @signalwire/js@4.0.0-rc.2 from the CDN, no wrapper, this page as-is. The page now imports 4.0.0-rc.3; re-pin the import in app.js to rc.2 to reproduce those runs exactly. Run I is on rc.3.

Setup

- node serve.mjs, open <https://localhost:8443>
- Token and destination: a v4 subscriber access token and /public/<room-name> from your SignalWire space. Both persist in localStorage
- Second participant: join the same room from another browser. Needed to say what the room actually heard, and for run H it is the control: with its microphone muted its readout shows which tones are genuinely in the room
- DevTools console: **Verbose** on, **Preserve log** on. Right-click, **Save as...** when done
- Reload between runs. One run per log

The runs

Run	Open	Do	Log
A	/	Join. Share screen, cancel the picker . Share screen again, Chrome Tab, tick "Also share tab audio" . Wait 30s	run-A-screenshare-audio-fatal.log
B	/	Join. Add extra audio track . Wait 30s	run-B-supplied-audio-fatal.log
B2	/	Run B again, after the page gained verbatim logging of inbound verto.mediaParams WebSocket frames	run-B2-raw-mediaparams-frames.log
C	/	Join. Add extra audio track x5, waiting for each outcome. Rejoin after every answered one	run-C-invite-always-answered.log
C2	/	Run C against an ad hoc room instead of a scheduled one	run-C2-instant-meeting-always-answered.log
E	/?nowarm=1	Reset site permissions first. Join. Leave the permission prompt 8s , then Allow	run-E-gum-inside-call-create-timeout.log
H	/?maintone=1	Join. Add extra audio track . Watch Received audio 30s. Remove extra leg , watch 10s. Add extra audio track again. Meanwhile read the second browser's readout	run-H-own-additional-device-in-own-mix.log plus run-H-control-second-member.log
F	/?supportedapi=1	Join in two browsers. In one: Share screen and stop it (the control). Drop socket , wait for signal-only reconnect, toggle Mic , Share screen . While it waits, share from the other browser. After the 50s timeout toggle Mic again, then Leave	run-F-signal-only-reconnect-invite-refused.log

Run	Open	Do	Log
I	/	Join in two browsers and wait for two <code>verto.pings</code> , so the countdown shows. In one browser, once per reload: Drop socket as next ping lands , Drop socket just before next ping , Swallow next pong , and the control, Drop socket midway between pings. After each, wait at least two ping intervals and note whether a <code>verto.bye</code> arrives. Three times each	<code>run-I-drop-on-ping.log</code> , <code>run-I-drop-before-ping.log</code> , <code>run-I-swallow-pong.log</code> , <code>run-I-control-midway.log</code>
J	/	Run I's three lost-pong variants again with Resend the lost pong once recovered ticked. Same procedure: two browsers, two pings first, one variant per reload, wait at least two ping intervals. Three times each. The log ends each run with <code>SURVIVED</code> (a later <code>verto.ping</code> arrived) or <code>NOT RESCUED</code> (<code>verto.bye</code> came anyway)	<code>run-J-drop-on-ping.log</code> , <code>run-J-drop-before-ping.log</code> , <code>run-J-swallow-pong.log</code>

Runs C and C2 take a while: an *answered* invite destroys the call (the fatal-classification defect), so rejoin before the next attempt. An unanswered one would sit for the SDK's 50s timeout; none occurred in these captures.

Run E permission reset: `chrome://settings/content/siteDetails?site=https://localhost:8443`

There is no run for "Chrome hides the tab-audio checkbox when audio is not requested": that is correct, documented browser behaviour, not a defect. What matters is what happens when the box is offered and ticked, and run A covers that.

What each run proves

Run B / B2: supplied audio is stopped and the call is destroyed

```
supplied audio/... "MediaStreamAudioDestinationNode" live SUPPLIED:tone
track.stop() ... from ...replaceAudioTrackWithConstraints
getUserMedia({... "deviceId":{"exact":"WebAudio-..."}})
gUM REJECTED OverconstrainedError
[RTCPeerConnectionController] Destroying RTCPeerConnectionController. main
call destroyed
```

Chrome stamps a WebAudio track with a synthetic `WebAudio-<uuid>` device id, so the pinned re-capture always rejects. Public API only. The same file holds a control 25s earlier: the main leg's own swap, on a real microphone, succeeds.

B2 adds the raw wire frames, so the server-sent payload (`autoGainControl`, `echoCancellation`, `noiseSuppression`, and no `deviceId`) is read directly off the WebSocket rather than inferred from the `getUserMedia` constraints.

Run A: screen-share audio, same teardown, plus two controls

Same sequence with `deviceId: "web-contents-media-stream://..."`. The cancelled picker at the start is a control, not a bug: only the screenshare peer connection is destroyed and the call survives, which is the issue #5 fix working. Same classifier, right in one path and wrong in the other, 46 seconds apart in one file.

Note the failing leg is auxiliary and `main` is destroyed **first**, milliseconds before it.

Runs C / C2: invite timing, and the teardown every time

Each answered invite logs `ANSWERED in <n>ms`; every one landed between 5.6s and 5.9s, and every one then ended the call within about a second. C is a scheduled room, C2 an ad hoc one, so the behaviour is not room-

type specific.

Run F: new legs refused after a signal-only reconnect

First seen through another application, then reproduced on this page three times out of three on 2026-09-11 (refusals 214ms, 253ms and 253ms after the invite). The socket reconnects mid-call with the peer connection still up, `call.*` requests before and after the reconnect resolve, and the `verto.invite` for a new leg comes back with `-32003 Must provide a participant invite first`, followed by the 50s timeout.

```
screen share started                <- the control, before the drop
WebSocket #1 closed code=1000 reason="Close received" clean=true
WebSocket #2 open
CallRecoveryManager: signal-only reconnect ...
self.mute() resolved in <n>ms        <- member request served on the new socket
verto.invite frame to WebSocket: {...}
error frame from WebSocket: {"jsonrpc":"2.0","id":"...", "error":{"code":-32003,"message":"Must
provide a participant invite first","original_request":{"...}}}
screen share failed: TimeoutError: Timeout has occurred
```

The share before the drop is the control: same session, same room, same peer connection, and it succeeds. The first two runs predate the SDP redaction and the mic logging and are not shipped; the third is `run-F-signal-only-reconnect-invite-refused.log`. Its timings: `self.mute()` 310ms on the first socket; drop at 16:03:41.286, closed 16:03:41.511 with code 1000 "Close received", new socket open 16:03:42.078, `signal-only reconnect` 16:03:42.338; share at 16:03:45.260, invite 16:03:49.752, refusal 16:03:50.005; `TimeoutError` 16:04:35.267; `self.unmute()` on the new socket 16:05:29.330, resolved in 2758ms of which 2458ms was the `call.unmute` round trip. The SDK's own ICE-candidate debug lines carried the machine's public address four times; it is replaced with `PUBLIC_IP_REDACTED` and nothing else in the file is edited.

The `error` frame is the point of the run: it puts the JSON-RPC code next to the message, which no earlier capture managed. Run it at least three times. If the client-side close (code 1000) does not reproduce it, use a one-second wifi toggle instead, and if the log says `full reconnect` the peer connection dropped too and the attempt does not count.

Run H: a member's own additional device is in the mix sent back to it

Captured 2026-09-15, both browsers. A is `?maintone=1`, B is the muted control.

```
A 19:07:34.932 publishing a 660 Hz tone on this page's own leg, in place of the microphone
A 19:07:38.396 joined                                     (and no getUserMedia after the dial)
B 19:08:15.854 660 Hz (this page's own leg) IS IN ... peak -32.5 dB    <- A's tone is in the room
A 19:09:17.300 extra leg "tone 1" ANSWERED in 2575ms
A 19:09:17.824 child member from WebSocket:
{"member_id":"889f5bdf...", "parent_id":"3024199d...", "type":"device", "name":"Paul Taggart"}
A 19:09:18.104 440 Hz (this page's additional device) IS IN ... peak -32.1 dB, floor -143.3 dB
A 19:10:15.725 self.removeAdditionalDevice(58a9978f...) resolved in 564ms
A 19:10:16.852 440 Hz ... is gone ... peak -78.0 dB
A 19:10:29.343 extra leg "tone 2" ANSWERED in 2755ms
A 19:10:30.106 440 Hz ... IS IN ... peak -32.1 dB          <- and back again
```

`parent_id` on that member is A's own member id, which `call.joined` gave as `3024199d-a6d2-4729-b470-8eb011c567e7`. The server also named the device after A.

The numbers carry the argument. A's own 660 Hz never appears in A's own mix and sits at -32.5 dB in B's; A's child device's 440 Hz sits at -32.1 dB in A's own mix and -32.0 dB in B's. Same tone, same room, same level:

the child is mixed into its parent like any other member, and only the parent's own leg is excluded. Removing the leg takes it out and adding another puts it back, so the probe tracks the leg rather than the page.

Two things to check before trusting a run of your own:

- No `getUserMedia` after the dial in A. One means a capture device was opened after all, a microphone is in the room, and the control is not clean. There are none in the shipped log.
- B has to show both tones. If B shows nothing either, the mixer gated the sine rather than the probe missing it.

The shipped A log reports 660 Hz present twice for 500 ms, right after each child leg's audio enters the mix (peak `-55.1 dB`, floor `-112.9 dB`, then `-66.2 dB`, floor `-124.6 dB`). That is a broadband onset transient, not the tone: both are 23 dB or more below a real tone and both coincide with the spectral floor lifting around 30 dB. The probe now wants a full second of agreement instead of 500 ms, which rules it out. The logs predate that change.

Run I: one missed `verto.ping` ends the call

Not yet captured. Seen first through another application on rc.3, 2026-09-25: a signal-only reconnect whose gap covered a ping, then no `verto.ping` on the new socket, then `call.left` and `verto.bye` from the server 46 seconds later. Two reconnects between pings the same day did nothing. What each variant should show if that reading is right:

```
verto.ping #4 id=22130 on WebSocket #1, 46.0s after the last
dropping WebSocket #1 as verto.ping #4 lands, before the SDK can answer it
verto.pong for verto.ping #4 (...) sent on WebSocket #1 (CLOSING)  <- discarded by the browser
WebSocket #1 closed code=1000 ...
CallRecoveryManager: signal-only reconnect ...
verto.ping overdue: none for 51.0s, every 46.0s
call.left frame from WebSocket #2: {...}
verto.bye frame from WebSocket #2: {... "cause": ..., "causeCode": ...}
```

The three variants answer different questions, so a run of each is needed, not a pick:

- **Drop as it lands** reproduces the original sequence.
- **Drop just before** says whether a ping sent into the gap is ever re-delivered.
- **Swallow** takes the reconnect out. If that alone ends the call, one lost answer is fatal on any path, and a reconnect is just one way of losing it.
- **Midway** is the control: if it ends the call too, the ping is not the cause.

Real network drops close with 1006, not 1000. If the client-side close behaves differently, repeat the first variant with a wifi toggle of about a second timed from the countdown, and if the log says `full reconnect` the peer connection dropped too and the attempt does not count.

Run J: can a resent `verto.pong` save the call?

Run I showed one lost pong is fatal. Run J asks whether a client can work around that until the server is fixed, by sending the answer again once it can. The page copies the SDK's last real `verto.pong` frame and sends it with fresh request ids:

- after **Drop as it lands** and **Drop just before**, the moment the next socket's `signalwire.connect` is answered;

- after **Swallow**, 6s later on the same socket, just after the SDK's pong RPC times out and it raises `VertoPongError` (the moment a workaround would learn of the loss).

The copy fits any ping on the call: a pong does not name the ping it answers, and its params are the call's `callID` and `dialogParams` only. So **Drop just before**, where the page never sees the ping, is the telling variant: it asks whether the server accepts an answer to a ping the client never received.

```
WebSocket #4 authenticated (signalwire.connect answered)
RESENDING a verto.pong on WebSocket #4 (the connection is back after the drop), outer id=...
verto.pong for no pending verto.ping acknowledged by the server in 450ms
SURVIVED: a verto.ping arrived 45.1s after the resent verto.pong      <- workaround viable
or
NOT RESCUED: verto.bye 40.2s after the resent verto.pong            <- only the server can fix it
```

An acknowledged resend that is still followed by a bye means the server accepts the frame but does not count it against the missed ping.

Run E: the 6s call-create timeout contains `getUserMedia`

```
getUserMedia({"audio":{},"video":{}})
... 6.00s ...
[Session] Error creating outbound call: TimeoutErrorImpl
join failed: CallCreateError: Call create timeout
... after you click Allow ...
InvalidStateError: ... signalingState is 'closed'
```

A slow permission prompt and a slow camera are the same event to this timer. The capture is not cancelled on timeout: the devices open for a call that no longer exists.

Flags

Flag	Effect
<code>?nowarm=1</code>	skips the pre-dial device warm-up, which otherwise moves the capture outside the 6s budget and hides the timeout defect
<code>?supportedapi=1</code>	shares via <code>startScreenShare()</code> instead of the private <code>vertoManager</code> , so the share carries no audio and cannot trigger runs A or B. Used by run F
<code>?maintone=1</code>	dials with a 660 Hz tone in place of the microphone, so the page publishes a known tone on its own leg and opens no capture device at all. Used by run H

All off by default, all announced in the log at startup, so every log records which mode produced it.